



**UNIVERSITE DE FIANARANTSOA
ECOLE NATIONALE D'INFORMATIQUE**

**Mention : INTELLIGENCE ARTIFICIELLE
Parcours : OBJET CONNECTE ET CYBERSECURITE**

Intitulé :

Tutoriel - Sécuriser DVWA avec ModSecurity et OWASP CRS

Installation, configuration et validation d'un WAF sur Kali Linux

Année universitaire: 2025-2026

Table des matières

Introduction	3
Étape 1 - Préparer l'environnement.....	4
Étape 2 - Installer DVWA	4
Étape 3 - Constater l'exposition initiale (sans WAF)	5
Test A - Injection SQL	5
Test B - Cross-Site Scripting (XSS).....	5
Étape 4 - Installer ModSecurity et le CRS	5
Activer le mode blocage.....	6
Vérifier l'inclusion du CRS.....	6
Redémarrer Apache.....	6
Étape 5 - Revalider les mêmes attaques (avec WAF)	6
Test A - Injection SQL (avec CRS)	6
Test B - XSS (avec CRS)	7
Consulter le log d'audit	7
Étape 6 - Dépannage (erreurs fréquentes)	7
Aller plus loin.....	8
Rappel de sécurité	8

Introduction

Ce tutoriel accompagne le dépôt Git du projet. Il détaille, étape par étape, comment :

1. déployer DVWA (Damn Vulnerable Web Application) comme cible de test volontairement vulnérable ;
2. valider son exposition avec deux attaques simples (injection SQL, XSS) ;
3. déployer ModSecurity et le jeu de règles OWASP Core Rule Set (CRS) comme pare-feu applicatif (WAF) ;
4. revalider les mêmes attaques pour constater leur blocage.

Prérequis : une machine Kali Linux (physique ou virtuelle), isolée du réseau public, avec un accès sudo.

 **Avertissement de sécurité** - DVWA est une application volontairement truffée de failles. Elle ne doit **jamais** être exposée sur Internet ni sur un réseau partagé avec des machines de production. Utilisez une machine virtuelle isolée (VirtualBox, VMware) en réseau interne.

Étape 1 - Préparer l'environnement

Mettre à jour le système et installer les paquets de base :

```
sudo apt update  
sudo apt install apache2 mariadb-server php php-mysqli php-gd libapache2-mod-php git
```

Démarrer les services :

```
sudo systemctl enable apache2 mariadb  
sudo systemctl start apache2 mariadb
```

Vérifier qu'Apache répond sur `http://127.0.0.1/`.

Étape 2 - Installer DVWA

Cloner le dépôt officiel directement dans la racine web d'Apache :

```
cd /var/www/html  
sudo git clone https://github.com/digininja/DVWA.git dvwa  
cd dvwa  
sudo cp config/config.inc.php.dist config/config.inc.php
```

Éditer le fichier de configuration :

```
sudo nano config/config.inc.php
```

Renseigner les identifiants de connexion à la base de données (`db_user`, `db_password`, `db_database`)
- ils doivent correspondre à un utilisateur MariaDB existant ou à créer.

Attribuer les droits nécessaires aux fichiers :

```
sudo chown -R www-data:www-data /var/www/html/dvwa  
sudo chmod -R 755 /var/www/html/dvwa
```

Ouvrir un navigateur vers :

`http://127.0.0.1/dvwa/setup.php`

Cliquer sur **Create / Reset Database**. DVWA crée alors sa base et ses tables automatiquement.

Se connecter avec les identifiants par défaut (admin / password), puis dans **DVWA Security**, régler le niveau de difficulté sur **Low** pour ce tutoriel (les payloads utilisés plus bas sont calibrés pour ce niveau).

Étape 3 - Constater l'exposition initiale (sans WAF)

Test A - Injection SQL

Dans le module **SQL Injection** de DVWA, saisir dans le champ User ID :

```
1' OR '1'='1
```

Résultat attendu sans protection : la requête retourne l'ensemble des utilisateurs de la base, alors qu'un seul était demandé - preuve d'une injection SQL réussie par tautologie.

Test B - Cross-Site Scripting (XSS)

Dans le module **XSS (Reflected)**, saisir dans le champ de saisie :

```
<script>alert(1)</script>
```

Résultat attendu sans protection : une pop-up JavaScript s'affiche - preuve que le script injecté a été exécuté par le navigateur.

À ce stade, l'application est totalement exposée : c'est la baseline à comparer après activation du WAF.

Étape 4 - Installer ModSecurity et le CRS

Installer ModSecurity et le paquet CRS packagé pour Apache :

```
sudo apt install libapache2-mod-security2 modsecurity-crs
```

Ce paquet :

- installe le moteur ModSecurity v3 ;
- active automatiquement le module Apache security2 ;

- fournit les règles du OWASP Core Rule Set.

Activer le mode blocage

Par défaut, ModSecurity peut être en mode détection seule. Éditer

/etc/modsecurity/modsecurity.conf et s'assurer que la ligne suivante est présente :

```
SecRuleEngine On
```

(DetectionOnly journalise sans bloquer - utile en phase de test, mais insuffisant pour une protection active.)

Vérifier l'inclusion du CRS

Vérifier que les règles CRS sont bien chargées par Apache. Selon la distribution, cela se trouve dans /etc/apache2/mods-available/security2.conf ou /etc/apache2/conf-available/modsecurity.conf :

```
IncludeOptional /usr/share/modsecurity-crs/*.conf
```

```
IncludeOptional /usr/share/modsecurity-crs/rules/*.conf
```

Redémarrer Apache

```
sudo systemctl restart apache2
```

```
sudo systemctl status apache2
```

Si le service redémarre sans erreur, la configuration est syntaxiquement valide et ModSecurity est actif.

Étape 5 - Revalider les mêmes attaques (avec WAF)

Relancer exactement les deux tests de l'étape 3, sans rien changer côté DVWA.

Test A - Injection SQL (avec CRS)

Même payload : 1' OR '1'='1

Résultat attendu : la page affiche une erreur **403 Forbidden** au lieu du résultat de la requête. La requête n'atteint jamais le code applicatif de DVWA.

Test B - XSS (avec CRS)

Même payload : `<script>alert(1)</script>`

Résultat attendu : la balise `<script>` est détectée et la requête bloquée - aucune pop-up ne s'affiche.

Consulter le log d'audit

```
sudo tail -f /var/log/apache2/modsec_audit.log
```

Un exemple de ligne de log attendue pour l'injection SQL :

```
[modsecurity] [client 192.168.1.10] Access denied with code 403
```

```
Message: SQL Injection Attack Detected (Tautology logic check)
```

```
Rule ID: 941100 -- Severity: CRITICAL
```

```
Action: Intercepted (Transaction Dropped)
```

Le Rule ID permet d'identifier précisément quelle règle du CRS a déclenché le blocage - utile pour ajuster la configuration en cas de faux positif.

Étape 6 - Dépannage (erreurs fréquentes)

Symptôme	Cause probable	Solution
Apache refuse de redémarrer après activation de ModSecurity	Erreur de syntaxe dans un fichier .conf inclus	<code>sudo apachectl configtest</code> pour localiser la ligne fautive
Aucun blocage constaté malgré l'installation	SecRuleEngine encore sur DetectionOnly ou Off	Repasser sur On dans <code>modsecurity.conf</code> , puis redémarrer Apache
DVWA affiche une erreur de connexion à la base	Identifiants incorrects dans <code>config.inc.php</code>	Vérifier <code>db_user</code> / <code>db_password</code> / <code>db_database</code> et l'utilisateur MariaDB correspondant
Un test légitime de l'application est bloqué à tort	Faux positif du CRS (niveau de paranoïa trop strict)	Consulter le Rule ID dans le log, puis créer une exception ciblée plutôt que de désactiver tout le CRS

Aller plus loin

- Documenter les faux positifs rencontrés et créer des règles d'exception ciblées plutôt que de baisser le niveau de sécurité global.
- Tester d'autres modules DVWA (Command Injection, File Upload, CSRF) avec la même méthodologie avant/après.
- Comparer le comportement du CRS selon son niveau de paranoïa (PL1 à PL4), qui règle le compromis entre détection et faux positifs.

Rappel de sécurité

Ce laboratoire est destiné à un usage pédagogique en environnement isolé. Ne jamais :

- exposer DVWA sur un réseau accessible depuis Internet ;
- réutiliser les identifiants par défaut de DVWA (admin / password) en dehors de ce contexte de test ;
- désactiver ModSecurity sur un serveur de production pour "gagner en performance" sans évaluer le risque.